ZenteiQ MaxText Assignment Report

CPU, GPU, and TPU comparison for dense Qwen and DeepSeek MoE synthetic training runs

Repository: https://github.com/exkzit-code/zenteiq-assignment-15062026

Generated from repo metrics: 2026-06-20

Assignment Coverage

PDF requirement	Repository evidence	Status
Task 1: MaxText data formats	docs/task1_data_formats.md	Complete
Task 2: Qwen 0.6B and scaled dense model	configs/runs.json, configs/model_notes.md, raw logs	Complete
Task 2: 50 steps on CPU, GPU, TPU	3 Qwen base logs and 3 scaled Qwen logs	Complete
Task 3: DeepSeek MoE under 1B	MoE config overrides and 3 raw logs	Complete
All possible metrics	results/metrics/step_metrics.csv, run_summary.csv, config_params.json	Complete
Final consolidated interpretation	docs/final_analysis.md and this report	Complete

Executive Summary

All nine required MaxText runs completed: three architectures/model variants across CPU, GPU, and TPU, with 50 training steps per run and synthetic data throughout.

The comparison is apples-to-apples within each model: the same notebook, run matrix, model overrides, synthetic data setting, and 50-step target were used; only the backend changed.

CPU was slowest, GPU was hundreds of times faster than CPU, and TPU was fastest in these Colab runs. Scaling Qwen from 596M to 981M parameters slowed all backends, as expected. The DeepSeek MoE model was scaled under 1B reported total parameters at 478M and showed separate MoE metrics.

Task 1 - MaxText Data Formats

dataset_type	What it is	Strengths	Tradeoffs
synthetic	Generated token batches	Best for backend/model benchmarking; removes data loading variance	Does not test real data preprocessing or learning quality
hf	Hugging Face datasets pipeline	Convenient for public datasets and fast iteration	Can depend on network/cache behavior; less ideal for deterministic large-scale runs
grain	Grain data pipeline	Strong large-scale option; supports ArrayRecord and Parquet	More setup work; requires prepared data files
tfds	TensorFlow Datasets / TFRecord-style pipeline	Stable ecosystem when data is TFDS-compatible	Less flexible for some LLM workflows; tokenizer and prep constraints can matter

The assignment runs use **dataset_type=synthetic**. This is required by Tasks 2 and 3 and is the correct benchmarking choice because it isolates model and backend behavior from dataset I/O and preprocessing effects.

Model Configuration Summary

Field	Qwen3 0.6B base	Scaled Qwen approx 1B	Reason
base_emb_dim	1024	1280	Wider hidden state and projections
base_num_query_heads	16	20	Keeps attention shape aligned with hidden size
base_num_kv_heads	8	10	Keeps grouped-query ratio consistent
base_mlp_dim	3072	3840	Scales feed-forward capacity
base_num_decoder_layers	28	32	Adds depth to approach 1B parameters
reported parameters	596M	981M	Source of truth: MaxText logs

Field	DeepSeek2 16B starting point	Scaled-down MoE	Reason
base_emb_dim	2048	1024	Cuts hidden width
base_num_decoder_layers	27	12	Reduces depth
num_experts	64	8	Major reduction in total MoE parameters
num_experts_per_tok	6	2	Keeps routed top-k behavior while reducing active compute
shared_experts	2	1	Retains shared expert behavior
base_moe_mlp_dim	1408	768	Reduces each expert feed-forward block
reported parameters	16B family	478M	Under 1B requirement satisfied

Run Summary - Last Step Metrics

Run	Backend	Arch	Status	Steps	Sec	TFLOP/s/dev	Tok/s/dev	Loss	Perplexity	MoE lb	Params
dense_qwen_0_6b_cpu	cpu	dense	complete	50	7.464	0.015	4.287	0	1	-	596M
dense_qwen_0_6b_gpu	gpu	dense	complete	50	0.023	4.952	1380.56	0	1	-	596M
dense_qwen_0_6b_tpu	tpu	dense	complete	50	0.014	8.3	2313.978	0	1	-	596M
dense_qwen_scaled_1b_cpu	cpu	dense	complete	50	11.801	0.016	2.712	0	1	-	981M
dense_qwen_scaled_1b_gpu	gpu	dense	complete	50	0.033	5.728	970.638	0	1	-	981M
dense_qwen_scaled_1b_tpu	tpu	dense	complete	50	0.025	7.619	1291.051	0	1	-	981M
moe_deepseek_under_1b_cpu	cpu	moe	complete	50	4.672	0.01	6.849	0.264	1.303	0	478M
moe_deepseek_under_1b_gpu	gpu	moe	complete	50	0.015	2.969	2137.466	0.262	1.299	0	478M
moe_deepseek_under_1b_tpu	tpu	moe	complete	50	0.01	4.328	3116.175	0.268	1.307	0	478M

This table is generated from results/metrics/run_summary.csv. Full per-step metrics for all 450 steps are in results/metrics/step_metrics.csv.

Backend Speedup

Model	GPU tokens/sec vs CPU	TPU tokens/sec vs CPU	TPU tokens/sec vs GPU	CPU / GPU / TPU last seconds
Qwen3 0.6B	322.0x	539.8x	1.68x	7.464 / 0.023 / 0.014
Qwen3 approx 1B	357.9x	476.1x	1.33x	11.801 / 0.033 / 0.025
DeepSeek MoE under 1B	312.1x	455.0x	1.46x	4.672 / 0.015 / 0.01

Memory Summary

Model	CPU total GB	GPU total GB	TPU total GB	Observation
Qwen3 0.6B	7.2	5.0	4.5	Base dense model footprint
Qwen3 approx 1B	11.7	8.5	8.4	Memory rises with width and depth
DeepSeek MoE under 1B	5.2	3.7	3.7	Lower reported total memory than dense scaled model

Interpretation

CPU vs GPU vs TPU. The hardware ordering is consistent. CPU is slowest because this workload is dominated by parallel tensor operations. GPU is much faster due to high-throughput matrix kernels and memory bandwidth. TPU is fastest here because the runs use bfloat16-friendly tensor shapes on a TPU runtime.

Base Qwen vs scaled Qwen. The scaled Qwen run increased reported parameters from 596M to 981M. Step time rose on every backend: CPU 7.464s to 11.801s, GPU 0.023s to 0.033s, and TPU 0.014s to 0.025s. This matches the larger hidden size, MLP width, attention heads, and layer count.

Dense vs MoE. The DeepSeek MoE run reports 478M total parameters and uses routed experts. MoE total parameter count and active compute are different concepts: only selected experts are active per token. At this small batch and sequence length, accelerator throughput is strong, but routing behavior and MoE-specific metrics still matter.

Unexpected Findings and Caveats

Finding	Impact	Handled how
Dense Qwen logs show loss 0.0 and perplexity 1.0 on all backends	Do not treat dense loss as meaningful convergence	Documented as a metric anomaly; throughput comparison remains valid because behavior is backend-consistent
GPU setup hit Colab cuDNN vs Transformer Engine conflict	Initial GPU run failure	GPU setup removes optional Transformer Engine for Colab
TPU setup required an actual TPU runtime	Backend variable alone was not sufficient	Runner checks accelerator visibility before MaxText
TPU MaxText 0.2.2 hit TensorFlow/JAX LLVM option collision	Training aborted before step 0	For synthetic TPU runs, setup patches installed MaxText to avoid unnecessary TensorFlow imports

Evidence Map

Evidence	Path
Run matrix and config overrides	configs/runs.json
Model scaling rationale	configs/model_notes.md
Task 1 data format notes	docs/task1_data_formats.md
Raw MaxText logs	results/raw_logs/*.log
All per-step metrics	results/metrics/step_metrics.csv
Run-level metrics	results/metrics/run_summary.csv and run_summary.md
Config snapshots parsed from logs	results/metrics/config_params.json
Commands used for each run	results/metrics/run_commands.jsonl

Conclusion

The submission satisfies the PDF requirements: all requested backends, both dense Qwen runs, the scaled-down DeepSeek MoE run, raw logs, all parsed metrics, config explanations, and consolidated interpretation are present. The GitHub repository remains the source of truth; this PDF is a standalone summary that points to the underlying evidence.